

Justin's Portfolio

Justin Burton

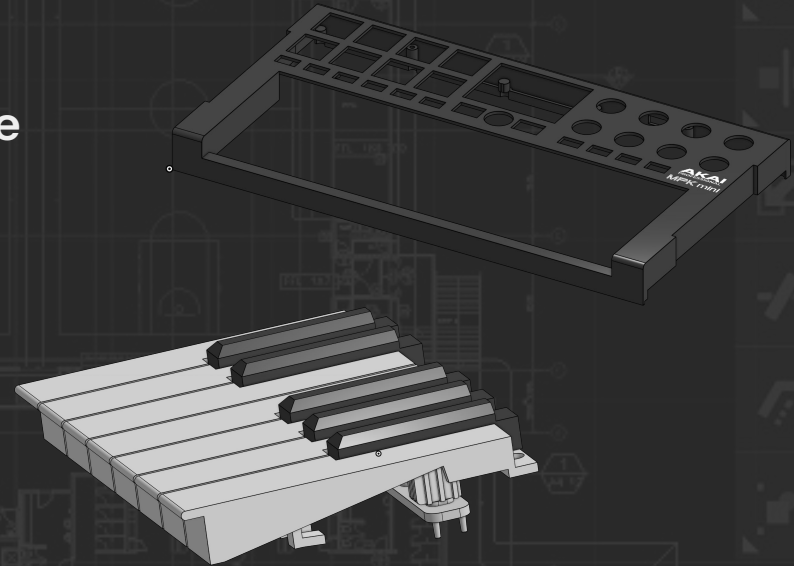
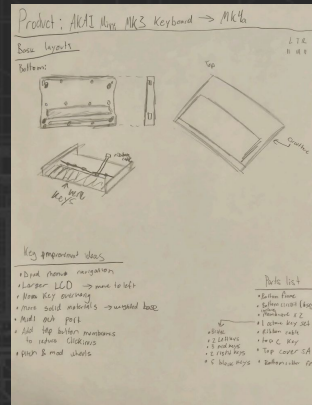
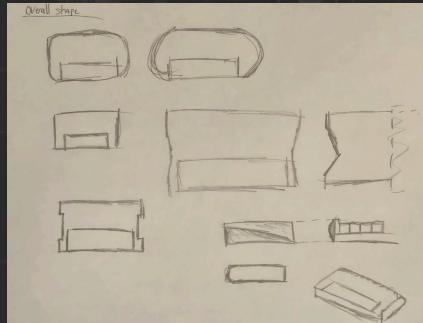
Justinrburton4@gmail.com | R1



AKAI MPK Mini MK3 Next Generation Product

Task: Evaluate feedback, ratings, and reviews of the AKAI MPK Mini MK3 and develop a next generation product implementing improvements.

Process: Sketch concepts, develop parts in CAD, import electronic components, create assembly



CAD Analysis

Interference detection

Instances to analyze

473340001 <5>	×
74LVC245 <22>	×
74LVC245 <2>	×
74LVC245 <8>	×
74LVC245 <12>	×
74LVC245 <14>	×
74LVC245 <10>	×
74LVC245 <18>	×
74LVC245 <3>	×
74LVC245 <19>	×
74LVC245 <15>	×
74LVC245 <16>	×
74LVC245 <21>	×

☐ Include standard content

☐ Show top level only

Interferences

EVW-AE4001B14 <6> and ...

UpperPCB <1> and 1770 <4>

UpperPCB <1> and SS-524...

↺

?

Part Face

Instances to measure

473340001 <5>	×
74LVC245 <22>	×
74LVC245 <3>	×
74LVC245 <18>	×

Mate connector for reference frame

☐ Show calculation variance

Mass

☒ Override

1.815 lb

Volume

41.107 in³

Surface area

1048.024 in²

Center of mass

☒ Override

X ↘

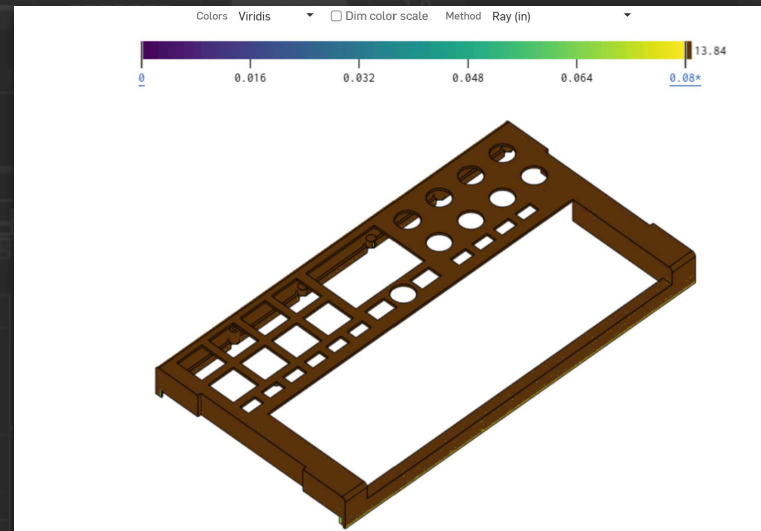
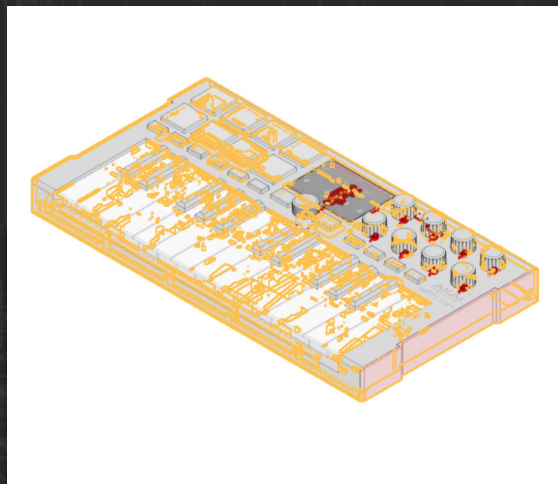
6.941 in

Y ↗

4.299 in

Z ↑

0.729 in



Measure

Select entities to measure

Face of Potentiometer <1>	×
Face of Base <1>	×

Measure type: Show all

Length unit: Inch

Angle unit: Degree

☐ Reference coordinate system

Parallel dist: 1.680 in

X ↘ 0.000 in

Y ↗ 0.000 in

Z ↑ 1.680 in

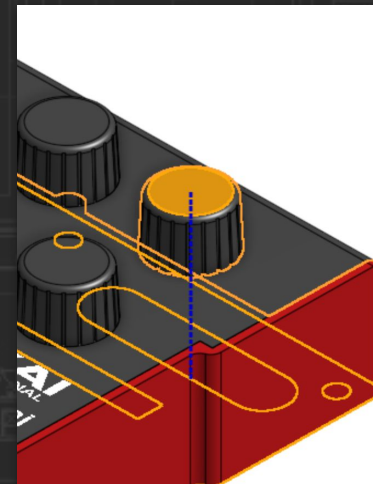
Area: 81.013 in²

Min dist: 1.680 in

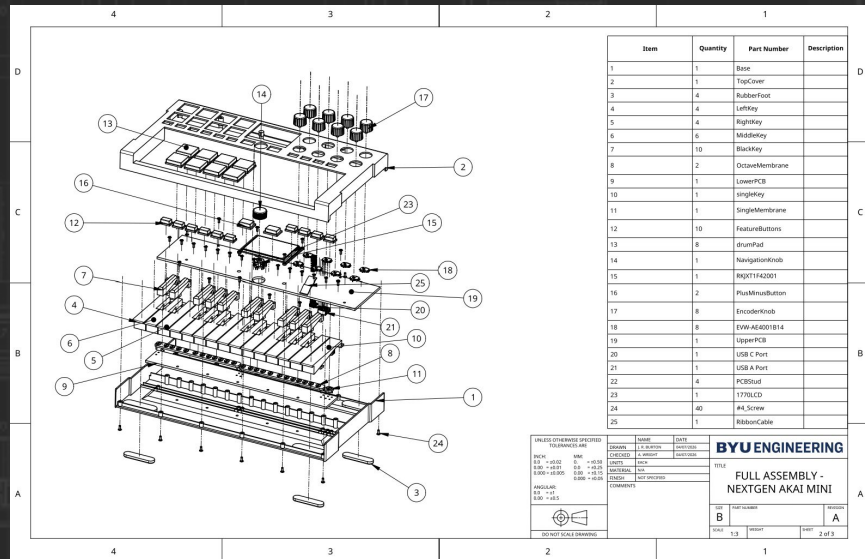
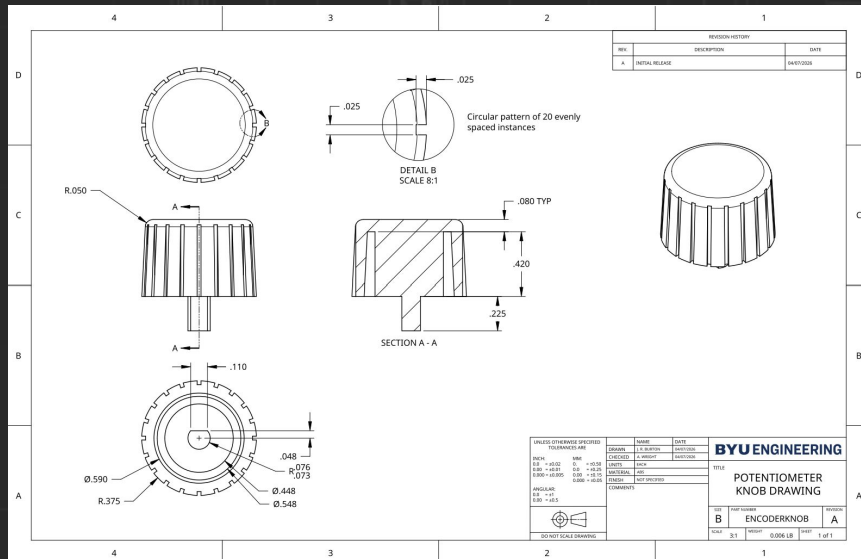
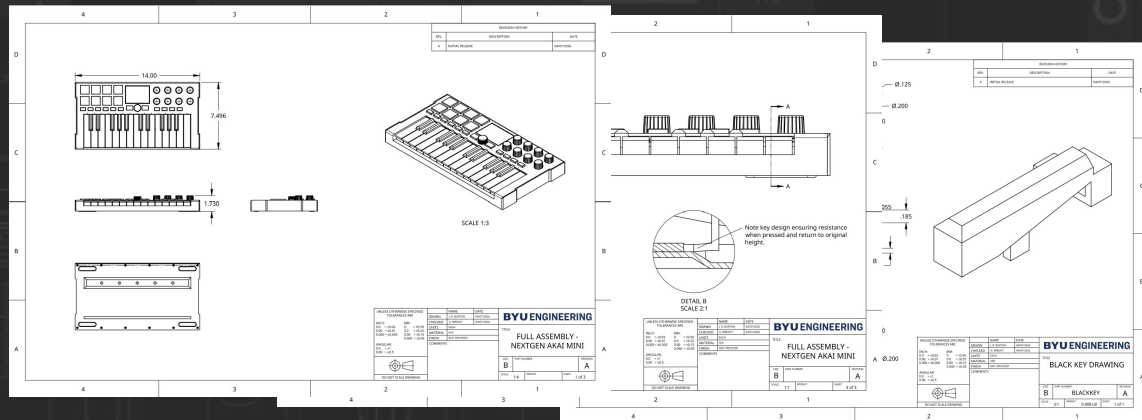
X ≠ 0.000 in

Y ≠ 0.000 in

Z ≠ 1.680 in



Engineering Drawings

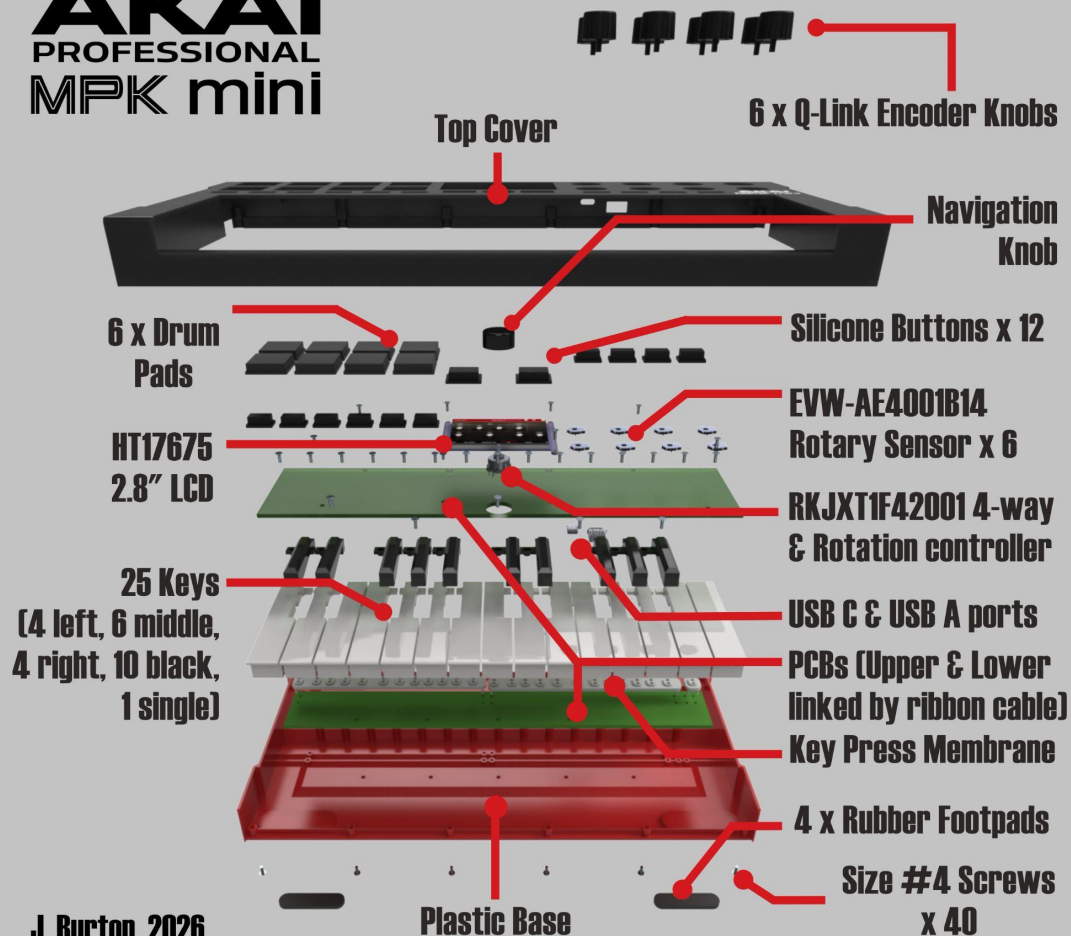


Professional Renders



USB ports, rotary sensors, LCD and
4-way controller imported

AKAI PROFESSIONAL MPK mini



J. Burton, 2026

OLD



Updated Intuitive layout

NEW



Larger LCD display

Improved navigation control

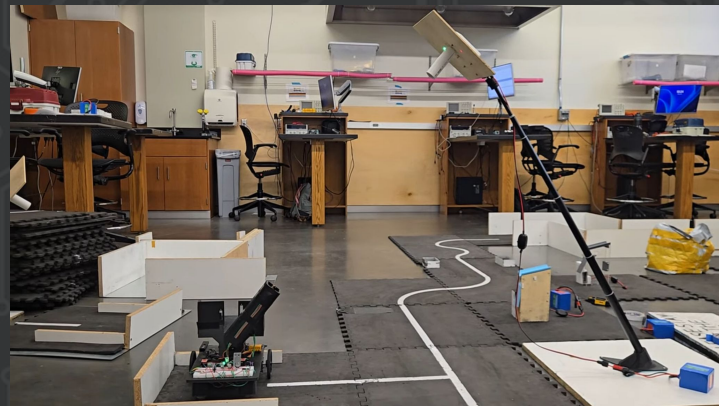
Sturdier Build

Autonomous Task-Specific Robot: Overview

Summary: Autonomous scale rover designed for unknown terrain navigation, trajectory following, and dynamic obstacle avoidance.

Firmware Architecture: Embedded C++ with Finite State Machines (FSM) for navigation modes, sensor processing, and emergency stops.

Motion Control: Closed-loop motor control logic and PWM drivers for precise differential steering and velocity regulation.

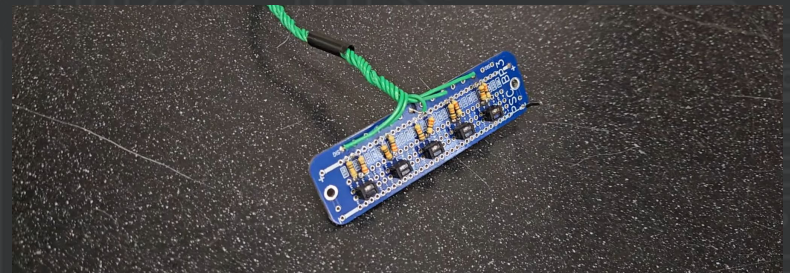
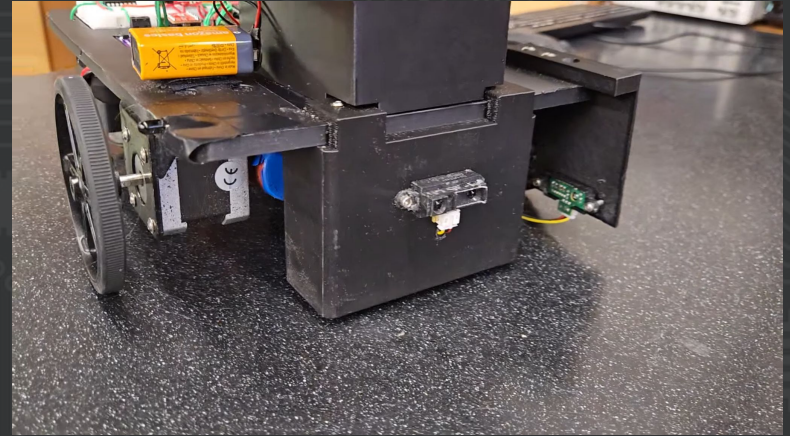


Autonomous Task-Specific Robot: Hardware & Testing

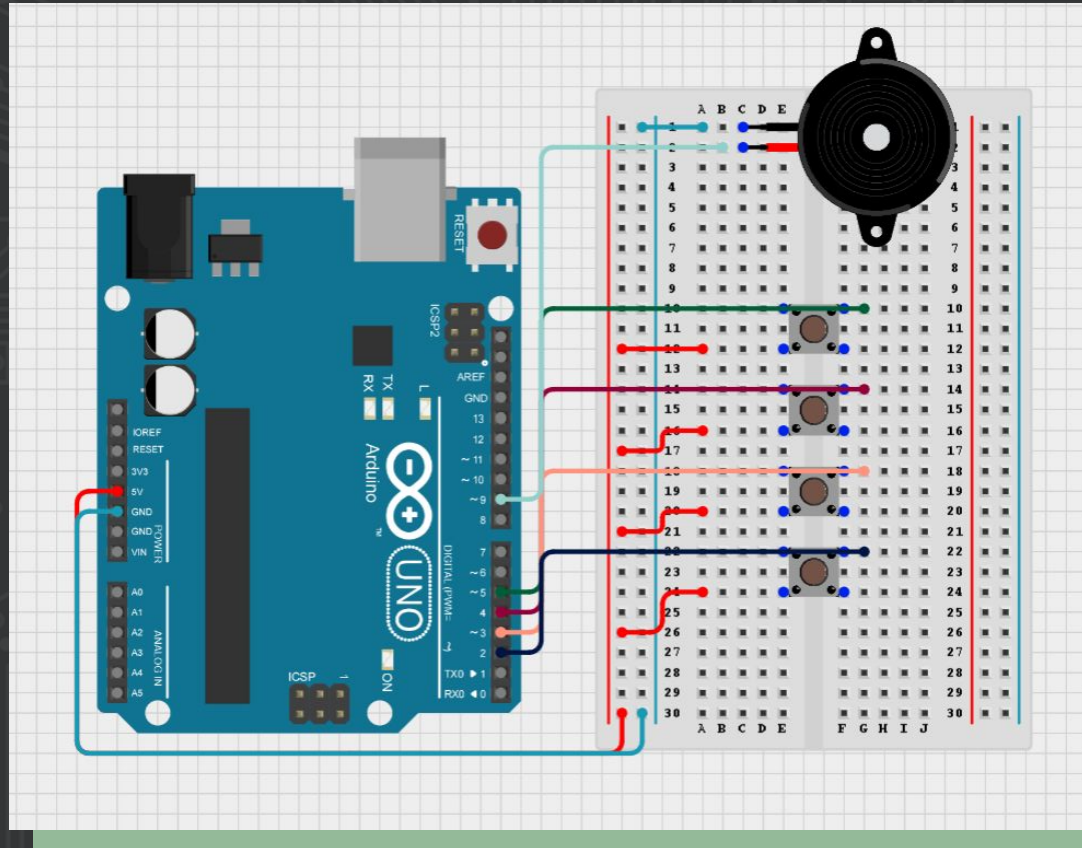
Sensor Array: Integrated optical and infrared distance modules for low-latency line-following and collision avoidance.

Custom Electronics: Designed custom hardware circuits and sensor interfaces.

Testing & Verification: Conducted Hardware-in-the-Loop testing and oscilloscope debugging to isolate power and signal noise.



One Key Keyboard



Parts List:

Arduino Uno
4 x Push button
Passive Buzzer
14 connector wires

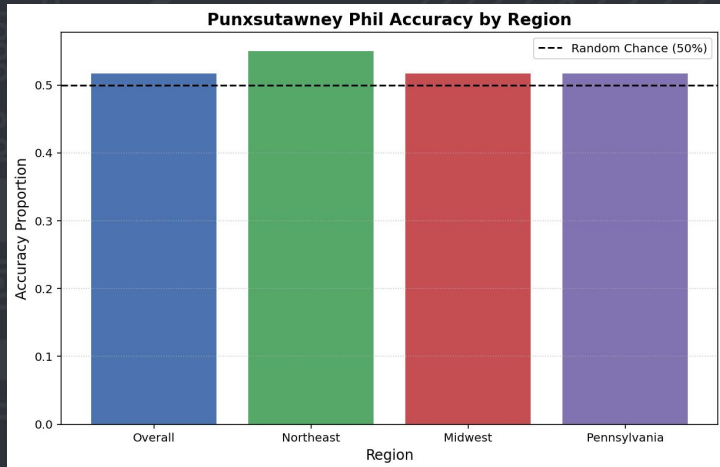
The focus:

Leveraging software
algorithms drive
complex sound design
on minimal and
resource-constrained
hardware.

Statistical Analysis

Goal: Evaluate data from the Punxsutawney Groundhog Club and NOAA to determine if the Groundhog Day groundhog's "weather forecasts" are more or less accurate than random chance

Spoiler: They are not



```
import pandas as pd
import numpy as np
from statsmodels.stats.proportion import proportions_ztest
import matplotlib.pyplot as plt

df = pd.read_csv('groundhogData.csv')

df = df[df['Punxsutawney Phil'] != 'No Record'].copy()

df['Feb_Mar_Avg'] = (df['February Average Temperature (Pennsylvania)'] +
                    df['March Average Temperature (Pennsylvania)']) / 2

threshold = df['Feb_Mar_Avg'].median()

df['Actual_Spring'] = np.where(df['Feb_Mar_Avg'] > threshold, 'Early Spring', 'Late Spring')

def check_accuracy(row):
    prediction = row['Punxsutawney Phil']
    actual = row['Actual_Spring']
    if (prediction == 'No Shadow' and actual == 'Early Spring') or \
        (prediction == 'Full Shadow' and actual == 'Late Spring'):
        return 1 # Correct
    return 0 # Incorrect

df['Is_Correct'] = df.apply(check_accuracy, axis=1)

# one tail test
successes = df['Is_Correct'].sum()
nobs = len(df)
null_prop = 0.5

#z stat & p value
z_stat, p_value = proportions_ztest(count=successes, nobs=nobs, value=null_prop, alternative='larger')

#95% conf
from statsmodels.stats.proportion import proportion_confint
ci_low, ci_high = proportion_confint(successes, nobs, alpha=0.05, method='normal')

print(f"Total Samples (n): {nobs}")
print(f"Correct Predictions: {successes}")
print(f"Sample Proportion: {successes/nobs:.4f}")
print(f"P-value: {p_value:.4f}")
print(f"95% Confidence Interval: ({ci_low:.4f}, {ci_high:.4f})")
```